At the time of writing, the actual syntax hasn't been decided and CoffeeScript doesn't currently support generators. Both the ECMAScript 6 specification and the CoffeeScript syntax to deal with it are still evolving.

A generator is like a function, but instead of just being evaluated, it can *suspend execution and resume later*. Being able to yield in the middle of a function means that the generator and the function that uses it work together. With regular functions, a function invocation always completes before the calling function resumes. In contrast, with generators, the calling function gets control back *many times*. That's why generator functions are sometimes known as *co-routines* in languages where the equivalents of functions are known as *routines*.

Will these generators be useful in CoffeeScript? Absolutely. They'll be useful in all those places where you've either ended up with event-emitter spaghetti code or have created your own abstraction to deal with them. Generators and comprehensions will give you more syntactic flexibility.

Language features in JavaScript are nice, but in the ever-expanding JavaScript universe, new languages are being invented almost daily. The rising popularity of compile-to-JavaScript languages (like CoffeeScript) means that runtimes need to find better ways to support them. One of the ways that JavaScript runtimes are starting to do this is with *source maps*.

13.4. Source maps for debugging

Debugging CoffeeScript by looking at the compiled JavaScript is difficult. An error in a running JavaScript program is easy to understand if the program was written in JavaScript but not so easy if the program was transpiled from another language such as CoffeeScript. So how do you use a debugger on a CoffeeScript program?

13.4.1. Why source maps?